

05 · Serielle Bussysteme

Mikrocomputertechnik 3 · Die Big 3 — Wie Mikrocontroller mit der Welt sprechen

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 80

Willkommen zu Block 5!

- Willkommen zur fünften Vorlesung in Mikrocomputertechnik 3
- Bisher: Programmierung und Hardware weitgehend *innerhalb* des Chips
- Auch PWM war vor allem Energie an einem Ausgangs-Pin
- Heute: physikalische Grenze des SoCs überschreiten
- Ziel: komplexe Daten mit externen Chips austauschen (Text, Messwerte, Konfiguration)

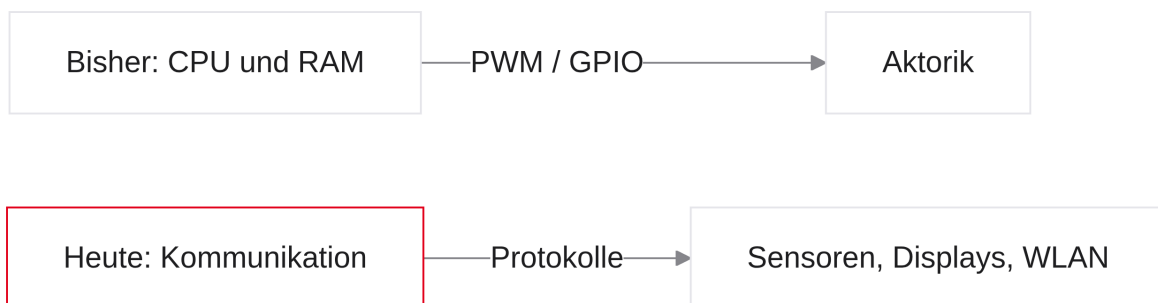
Notizen

Herzlich willkommen zurück! Speicher, Interrupts und Timer beherrschen Sie bereits. Moderne Systeme brauchen aber Sensoren, Displays und WLAN-Module. Wie transportieren wir Bits sicher über Kupferbahnen außerhalb des SoCs? Das ist das Thema heute.

Folie

2 / 80

Einordnung im Semester



Notizen

Von interner Steuerung zu On-Board-Kommunikation mit „smarten“ Peripheriebausteinen.

Anknüpfung: PWM und Watchdog

Kurz-Recap aus Block 4 (Vertiefung im Übungsblatt):

- Hardware-PWM: ARR → Frequenz, CCR → Duty Cycle
- Signal läuft jitterarm in der Peripherie, CPU orchestriert
- Watchdog: Freeze erkennen, harter Reset als Selbsthilfe
- Leitprinzip: harte Pin-Arbeit in Hardware, nicht per Bit-Banging

```
void main_loop(void) {  
    fade_led();  
    wdt_feed();  
}
```

Notizen

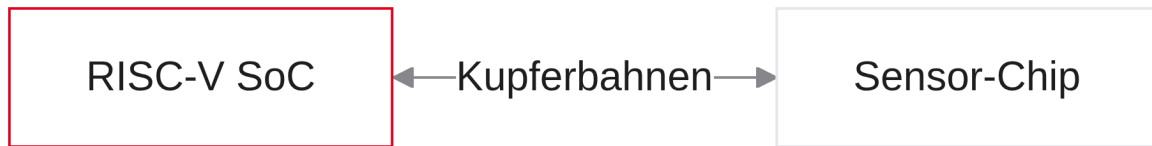
Behalten Sie bei: Die CPU orchestriert; Echtzeit-Pinarbeit gehört in dedizierte Module. Heute gilt dasselbe für Kommunikations-Hardware (UART/SPI/I2C-Controller).

Die SoC-Grenze

- Beispiel: digitaler Beschleunigungssensor (IMU) auf der Platine
- Sensor speichert Neigung (X, Y, Z) intern als Zahlen (z. B. 16 Bit)
- Frage: Wie kommen diese Bits in die Register/ALU unseres SoCs?
- Chips sind durch Kupferbahnen getrennt
- Wir brauchen ein physikalisches und zeitliches Regelwerk — ein Protokoll

Notizen

Innerhalb der CPU verschieben wir Worte über breite interne Busse. Zwischen zwei Chips zählen Laufzeiten, Störungen und gemeinsame Regeln.

SoC und Sensor — Schema**Notizen**

Schematisch: zwei getrennte Silizium-Bausteine, verbunden über die Platine.

Parallel vs. seriell

Parallele Übertragung:

- Viele Leitungen (z. B. 32) — ein Wort in einem Takt
- So arbeiten typische interne CPU–RAM-Pfade

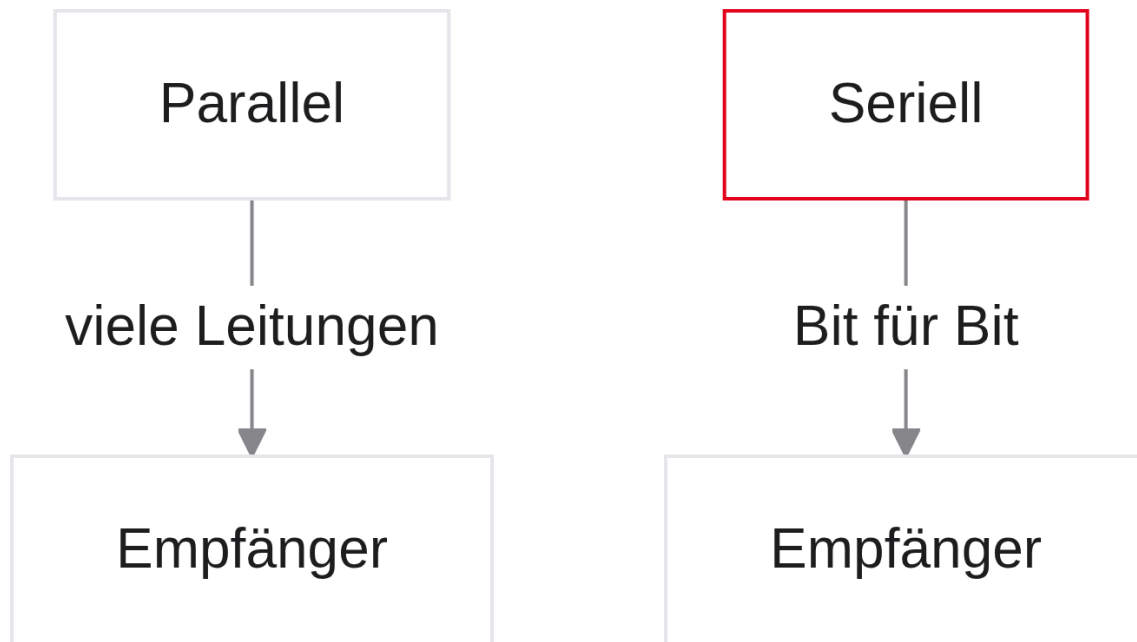
Serielle Übertragung:

- Wenige Leitungen (oft eine Datenleitung)
- Bits nacheinander — mehr Takte, weniger Pins

Notizen

Intuitiv wirkt parallel schneller. Extern haben sich dennoch serielle Standards durchgesetzt (USB, PCIe, On-Board-Busse). Warum?

Parallel vs. seriell — Schema



Notizen

Parallel: Breite. Seriell: Sequenz auf wenigen Leitungen.

Folie

8 / 80

Warum sich Seriell durchgesetzt hat

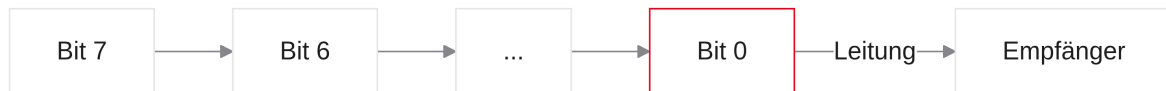
- Kosten und Platz: wenige Pins am Gehäuse sind günstiger
- Crosstalk: viele parallele Bahnen stören sich gegenseitig
- Clock Skew: bei hohen Frequenzen kommen Bits nicht mehr gleichzeitig an
- Seriell: sehr hohe Taktraten auf wenigen Leitungen möglich

Notizen

Bei hohen Frequenzen wirken parallele Bahnen problematisch (Störung, Laufzeitunterschiede). Seriell braucht mehr Takte, erlaubt aber oft höhere Bitraten am Ende.

Das Schieberegister

- Wandlung: parallele CPU-Daten → serieller Bitstrom
- Kern: Schieberegister (Kette von D-Flip-Flops, MCT1)
- CPU schreibt parallel; Kommunikations-Hardware schiebt Bit für Bit auf die Leitung
- Empfänger schiebt Bits wieder zu einem Wort zusammen



Notizen

UART-, SPI- und I2C-Controller basieren im Kern auf solchen Schieberegistern — parallel rein, seriell raus und umgekehrt.

Synchron vs. asynchron

Synchrone Kommunikation:

- Eigene Taktleitung (Clock, z. B. SCK)
- Empfänger tastet bei klaren Taktflanken ab — robust und schnell

Asynchrone Kommunikation:

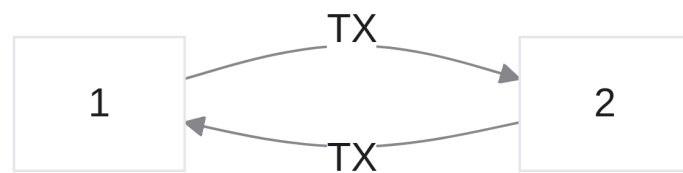
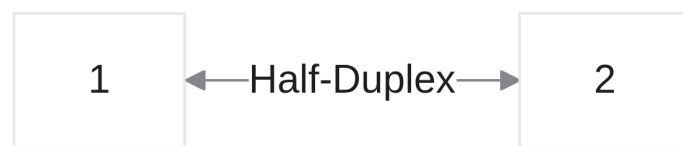
- Keine Taktleitung — nur Daten
- Beide Seiten brauchen vereinbarte Bitzeiten (Baudrate)

Notizen

Ohne Takt: Wie unterscheiden Sie drei Nullen von einer langen Null? Synchron: der Sender gibt den Schlag vor. Asynchron: gemeinsame „Stoppuhren“.

Simplex, Half-Duplex, Full-Duplex

- Simplex: nur eine Richtung (z. B. GPS sendet Positionen)
- Half-Duplex: beide Richtungen, aber nicht gleichzeitig (ein Medium)
- Full-Duplex: getrennte Pfade — gleichzeitig senden und empfangen



Notizen

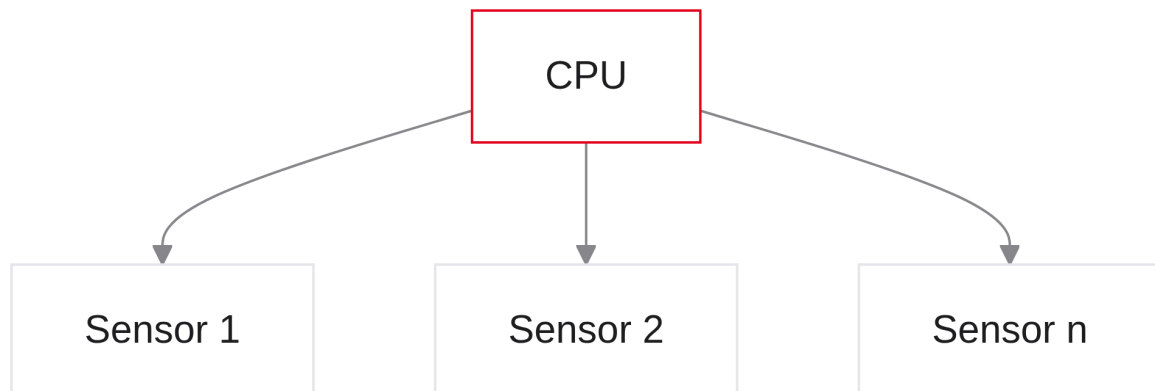
Wahlkriterium: Bandbreite vs. Pinzahl. Gelegentliche Sensorwerte → Half-Duplex oft genug; hohe Last → Full-Duplex.

Die Leitfrage des Tages

- Punkt-zu-Punkt ist einfach
- Moderne Platinen: viele Sensoren/Chips an einer CPU
- Nicht für jeden Teilnehmer eigene Leitungsbündel (Pin-Mangel)
- Wie teilen sich viele Geräte wenige Pins, ohne Chaos und Kurzschluss?

Notizen

Netzwerk auf der Platine: Wer spricht wann? Wie vermeiden wir gleichzeitiges Treiben derselben Leitung?

Leitfrage — Schema**Notizen**

Viele Teilnehmer, wenige Pins — das treibt UART vs. SPI vs. I2C.

Agenda: Die „Big 3“

Industrie-Standards für On-Board-Kommunikation:

1. UART — asynchron, Punkt-zu-Punkt
2. SPI — synchron, sehr schnell (Displays, Flash)
3. I2C — synchron, Netzwerk auf zwei Leitungen

	UART	SPI	I2C
Leitungen	2	4+	2
Takt	asynchron	synchron	synchron
Duplex	Full	Full	Half
Speed (Faust)	niedrig–mittel	hoch (SCK oft 10–50 MHz)	moderat (100–400 kHz)

Notizen

Alle drei finden Sie in typischen MCU-Datenblättern. Heute: wann welches Protokoll — Start mit UART.

Lernziele Block 5

- UART, SPI und I2C nach Pins, Takt und Topologie einordnen
- UART-Frame (Start/Stop/Parity) und Baudrate erklären
- SPI-Modi (CPOL/CPHA) und Chip-Select begründen
- I2C-Transaktion inkl. ACK/NACK und Repeated Start nachvollziehen
- Single-Ended vs. differenziell und EMI-Grundlagen einordnen

Notizen

Harris & Harris sowie Datenblatt-Kapitel zu Serial I/O — Protokollwahl ist Systemdesign.

Was ist UART?

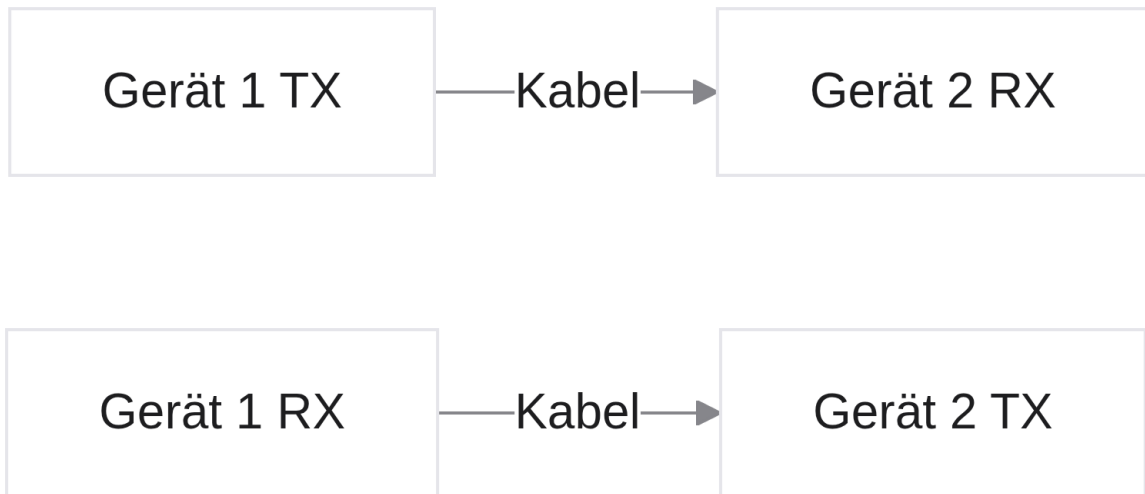
- UART: Universal Asynchronous Receiver-Transmitter
- Eher Schnittstelle/Direktverbindung als Multi-Drop-Bus
- Asynchron: kein Taktkabel
- Historisch: serielle Schnittstelle (RS-232, COM)
- Heute oft: Debug/print f über USB-UART-Wandler

Notizen

Mindestens ein UART-Controller ist in MCUs üblich. Terminal-Ausgabe läuft typischerweise über UART-Hardware.

Punkt-zu-Punkt

- Typisch genau zwei Teilnehmer
- TX (Transmit) und RX (Receive)
- Kreuzverkabelung: TX1 → RX2, TX2 → RX1
- Gemeinsames GND zwingend
- Full-Duplex auf zwei Datenleitungen



Notizen

Einfache Verkabelung, gleichzeitiges Senden möglich — aber keine dritte Station am selben Link.

Folie

18 / 80

Die Asynchronität

- Kein Taktkabel
- Problem: Ist HIGH eine Eins oder fünf Einsen hintereinander?
- Lösung: vereinbarte Bitdauer — gleiche Baudrate auf beiden Seiten

Notizen

Ohne Absprache der Bitzeit liest der Empfänger Müll. Die Baudrate ist die gemeinsame Stoppuhr.

Folie

19 / 80

Die Baudrate

- Baud: Symbole pro Sekunde; hier typisch 1 Bit pro Symbol → bps
- Klassiker: 9600 oder 115200 Baud
- Bei 115200 Baud: Bitdauer $\approx 1/115200 \approx 8,68 \mu s$
- Prescaler/Baud-Generator aus Systemtakt in C konfigurieren

Notizen

Sender und Empfänger müssen dieselbe Baudrate nutzen — sonst wirre Zeichen. 115200 ist heute oft Standard für Konsolen.

Der UART-Frame

- Idle: Leitung typisch HIGH
- Frame-Aufbau:
 - Start-Bit: LOW
 - Daten: meist 8 Bit (oft LSB first)
 - Parity: optional
 - Stop-Bit(s): HIGH

Abschnitt	Pegel / Inhalt
Idle	HIGH
Start	LOW
Data	8 Bits
Stop	HIGH

Notizen

Jedes Byte wird eingerahmt. Das Start-Bit markiert den Paketbeginn im Bitstrom.

Das Start-Bit

- Start-Bit (LOW): Synchronisations-Trigger
- Fallende Flanke Idle → Start startet die Baud-Uhr des Empfängers
- Abtastung typisch nahe der Bitmitte (erstes Datenbit nach $\approx 1,5$ Bitzeiten)
- Danach Abtastung im Raster von 1 Bitzeit; nach dem Frame Uhr stoppen

Notizen

Die fallende Flanke ist der Startschuss. Abtastung in der Bitmitte gibt dem Signal Zeit zur Stabilisierung.

Oversampling

- Empfänger tastet oft mehrfach pro Bit ab (z. B. 16×)
- Mehrheitsentscheid um die Bitmitte filtert kurze Spikes
- Erhöht Robustheit bei gestörten Leitungen

Notizen

Ein einzelner Samplepunkt wäre anfällig für Störimpulse. Oversampling mittelt/wählt Mehrheiten in der kritischen Zone.

Clock Drift und Stop-Bit

- Quarze weichen ab (Temperatur, Toleranz) → Clock Drift
- Lange ununterbrochene Ströme würden Abtastpunkte verschieben
- Start-Bit kalibriert neu; Stop-Bit (HIGH) stellt Idle wieder her
- Nächstes Start-Bit erzeugt wieder eine klare fallende Flanke

Notizen

Stop-Bits sind kein Luxus: Sie begrenzen Drift-Akkumulation auf Frame-Länge und sichern die nächste Synchronisation.

Das Parity-Bit

- Optional: Even oder Odd Parity über die Datenbits
- Empfänger prüft; bei Fehler oft Status/Interrupt (Framing/Parity Error)

Daten (Beispiel)	Einsen	Even	Odd
1010 0001	3	1	0
0000 0000	0	0	1

Notizen

Parity erkennt ungerade Bitfehler; doppelte Kipper können unbemerkt bleiben. Für hohe Sicherheit: stärkere Prüfsummen in Software.

FIFOs im UART

- Bei 115200 Baud und 8N1 (10 Bit/Frame): grob alle $\approx 87 \mu\text{s}$ ein Byte
- Interrupt pro Byte belastet die CPU stark
- Lösung: RX/TX-FIFOs (z. B. 16–64 Bytes), Interrupt bei Watermark



Notizen

Erinnerung Block 1: FIFOs entkoppeln. Die CPU holt Burst-weise Bytes aus MMIO, statt jedes Zeichen einzeln zu bedienen.

Nachteile von UART

- Keine Geräteadressen → schlecht skalierbar (kein Multi-Drop)
- Baudrate muss auf beiden Seiten übereinstimmen (starr konfiguriert)
- Framing-Overhead (Start/Stop/Parity) und asynchrone Grenzen
- Für Displays/Kameras/viele Sensoren meist ungeeignet

Notizen

UART eignet sich hervorragend für Debug und einfache Module — nicht als Hochleistungs-Netzwerkbus.

Zusammenfassung UART

Eigenschaft	UART
Leitungen	TX, RX (+ GND)
Takt	asynchron (Baudrate)
Duplex	Full-Duplex
Topologie	Point-to-Point (2 Teilnehmer)
Overhead	Start/Stop/(Parity)
Einsatz	Terminal, GPS, Bluetooth-Module

Notizen

Merksatz: UART = asynchron, 2 Leitungen, 2 Teilnehmer. Als Nächstes: synchron mit Taktleitung — SPI.

Folie

28 / 80

Fragen? — UART und Grundlagen

- Warum oft seriell statt parallel extern?
- Warum Start- und Stop-Bit?
- Welche Rolle spielen Hardware-FIFOs?

Notizen

Wenn Framing und Baudrate klar sind, führen wir den Takt explizit ein: SPI.

Folie

29 / 80

Was ist SPI?

- SPI: Serial Peripheral Interface (u. a. Motorola, 1980er)
- Synchron: eigene Taktleitung — kein Baudraten-Drift wie bei UART
- Full-Duplex, oft sehr hohe SCK-Taktraten (einige zehn MHz möglich; Bitrate $\approx$ SCK bei 1 Bit/Takt)
- Wenig Framing-Overhead: Bits folgen dem Takt

Notizen

Der Master gibt den Takt vor. Empfänger folgen dem Schlag — keine vorherige Baudraten-Einigung nötig.

Folie

30 / 80

Die Synchronität (SCK)

- Taktleitung: SCK (Serial Clock)
- Empfänger braucht keinen eigenen Baud-Generator
- Typisch: bei definierter Flanke Datenleitung abtasten
- Deterministisch, unempfindlich gegen Quarz-Abweichungen zwischen den Chips

Phase	Bedeutung
SCK-Flanke	Abtast-/Verschiebezeitpunkt

Notizen

Eine gemeinsame Uhr für Master und Slave — Uhren können nicht „auseinanderlaufen“ wie bei UART.

Folie

31 / 80

Die 4-Draht-Architektur

Klassisch 4 Signale (+ GND):

Signal	Richtung	Zweck
SCK	Master → Slave	Takt
MOSI	Master → Slave	Daten zum Slave
MISO	Slave → Master	Daten zum Master
CS / SS	Master → Slave	Slave auswählen (meist Active-Low)

Notizen

Kein UART-Kreuz: MOSI an MOSI-Logik, MISO entsprechend. Neuere Namen oft SDO/SDI — Konzept gleich.

Folie

32 / 80

Master-Slave

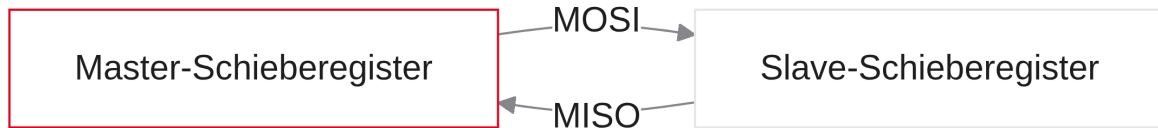
- Master (meist die MCU): erzeugt SCK, steuert CS, startet Transfers
- Slave: antwortet nur, wenn selektiert und getaktet
- Single-Master: ein aktiver Taktgeber auf dem Bus
- Slave „redet“ nicht von selbst

Notizen

Keine Buskollision durch unaufgefordertes Senden: der Master saugt/schiebt Daten aktiv.

Full-Duplex: Ringtausch

- Pro Takt: Bit auf MOSI hinaus und Bit auf MISO herein
- Nach n Takten: Schieberegister getauscht
- Auch zum reinen Lesen: Dummy-Bytes senden, damit SCK läuft



Notizen

SPI ist immer Tausch. Schreiben erzeugt Empfangsbytes (oft ignorieren); Lesen braucht Dummy-Writes.

Chip Select (CS)

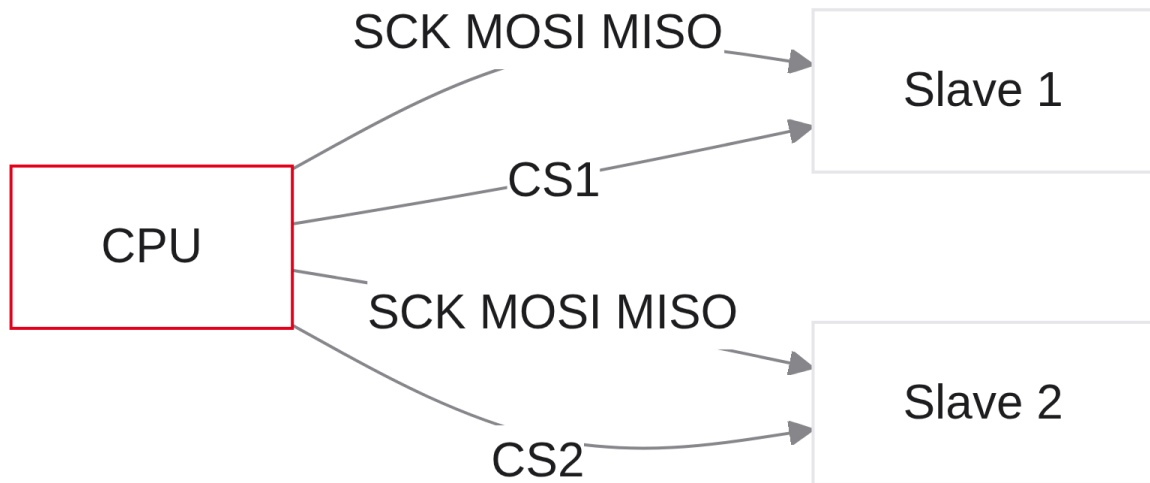
- SCK/MOSI/MISO parallel an mehrere Slaves möglich
- CS wählt den aktiven Slave (meist Active-Low)
- Andere Slaves: Bus ignorieren, MISO hochohmig

Notizen

CS ist das „Scheinwerferlicht“: nur der selektierte Teilnehmer treibt MISO.

Star-Routing

- Jeder Slave: eigenes CS → eigener GPIO am Master
- 10 Slaves $\approx$ 3 Busleitungen + 10 CS = viele Pins
- SPI skaliert bei vielen Teilnehmern pinseitig schlecht



Notizen

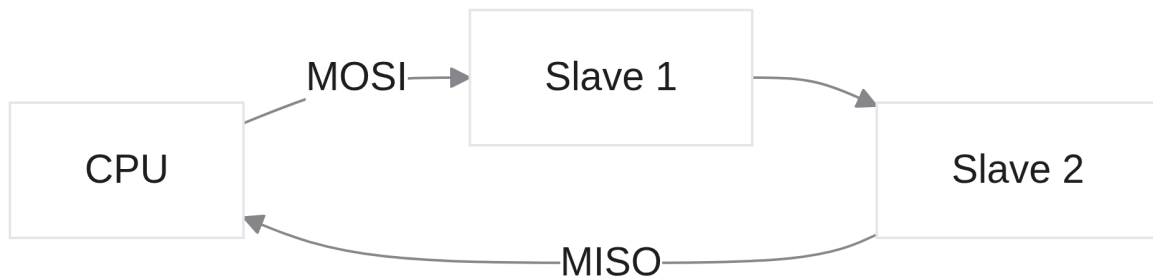
Geschwindigkeit hoch, Pinverbrauch ebenfalls — Alternative: Daisy-Chain oder anderer Bus (I2C).

Folie

36 / 80

Daisy-Chaining

- Ein CS für mehrere Schieberegister in Serie
- MOSI → Slave1 → ... → zurück MISO
- CPU schiebt lange Bitkette; Daten wandern durch die Kette
- Typisch bei LED-Ketten / Schieberegister-Arrays



Notizen

CS-Pins sparen, aber Updates betreffen die ganze Kette — für einzelne Slaves oft ineffizient.

Push-Pull und Geschwindigkeit

- SPI-Treiber typisch Push-Pull: aktiv HIGH und aktiv LOW
- Steile Flanken, schnelles Umladen der Leitungskapazität
- Ermöglicht hohe SCK-Frequenzen (je nach Layout und Slave)

Notizen

Im Kontrast dazu später I2C mit Open-Drain und Pull-ups — robuster für Multi-Drop, aber langsamer.

CPOL und CPHA

- Vier SPI-Modi (0–3): Kombination aus Ruhepegel und Abtastflanke
- CPOL: Idle LOW oder HIGH?
- CPHA: Abtastung bei erster oder zweiter Flanke?
- Immer Sensor-Datenblatt und Master-Register abgleichen — der Modus entscheidet über gültige Daten

Modus	CPOL	CPHA	Kurz
0	0	0	Idle LOW, Sample steigend
1	0	1	Idle LOW, Sample fallend
2	1	0	Idle HIGH, Sample fallend
3	1	1	Idle HIGH, Sample steigend

Notizen

Falscher Modus → Datenmüll trotz korrekter Verkabelung. Klassiker im Labor: alle vier Modi existieren; zwei Beispiele reichen nicht — Datenblatt lesen.

Nachteile von SPI

- Viele CS-Leitungen bei Star-Routing
- Nur physikalisches Bit-Schieben: keine Adressen, kein ACK in der Grundsicht
- Single-Master — Multi-Master schwierig
- Fehlende Slaves erkennt die Hardware nicht automatisch

Notizen

Software muss Plausibilität und Gerätepräsenz selbst prüfen.

Typische Einsatzgebiete

- Große Datenmengen, wenige Teilnehmer
- TFT-Displays, SD-Karten, externe Flash/EEPROM (auch Quad-SPI)
- Weniger ideal: dutzende langsame Sensoren (Pin-Overhead)

Notizen

SPI = Bandbreiten-Monster. Für viele kleine Sensoren kommt I2C.

Zusammenfassung SPI

Eigenschaft	SPI
Leitungen	SCK, MOSI, MISO, CS (+...)
Takt	synchron (Master)
Duplex	Full (Ringtausch)
Netz	Master/Slave, Star oder Daisy-Chain
Speed	hoch: SCK oft 10–50 MHz ($\approx$ Bitrate in Mbit/s bei 1 Bit/Takt)
Einsatz	Display, SD, Flash

Notizen

Als Nächstes: zwei Leitungen, Takt behalten, viele Adressen — I2C.

Folie

42 / 80

Fragen? — SPI

- Warum hilft eine Taktleitung gegenüber UART?
- MOSI/MISO und Ringtausch klar?
- Warum limitiert Star-Routing mit CS die Skalierbarkeit?

Notizen

Wenn Master, Takt und CS klar sind, folgt das Multi-Drop-Protokoll I2C.

Folie

43 / 80

Was ist I2C?

- I2C: Inter-Integrated Circuit (Philips/NXP, 1980er)
- Rückgrat vieler Sensor-Netzwerke auf der Platine
- Synchron: Taktleitung `SCL` (Serial Clock)
- Half-Duplex: eine Datenleitung `SDA` (Serial Data) — nicht gleichzeitig beide Richtungen
- Viele Bausteine an exakt zwei Signalen (+ GND)

Notizen

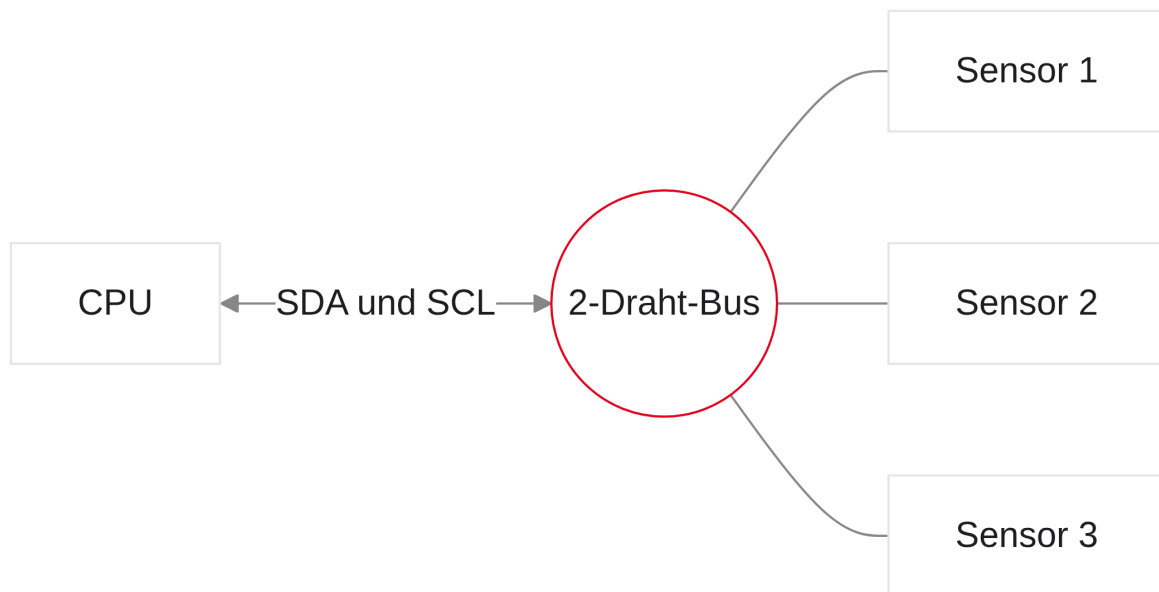
Auf Drohnen- oder Smartphone-Boards hängen Gyroskop, Barometer, Lade-IC und oft Touch am I2C — weil zwei Pins viele Geräte adressieren.

Folie

44 / 80

Ein echter Bus

- Kein SPI-Star-Routing mit vielen CS-Pins
- `SCL` und `SDA` als gemeinsame Busleitungen über die Platine
- Alle Teilnehmer parallel angeklemmt
- Dutzende Slaves an denselben zwei MCU-Pins möglich



Notizen

Echter Multi-Drop-Bus: gemeinsame Straße, geteilte Leitungen. Ohne CS: Adressierung in Software.

Folie

45 / 80

Adressierung statt CS-Kabel

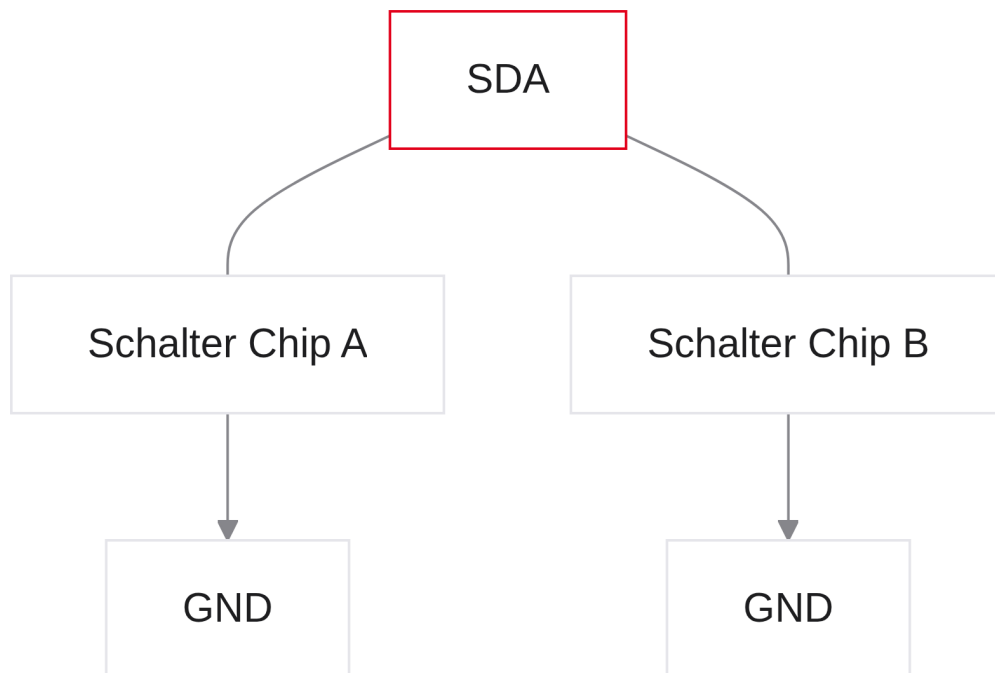
- Jeder Chip: feste Adresse (meist 7 Bit, z. B. 0×68 für MPU6050)
- Master sendet Adresse auf SDA
- Nur der angesprochene Slave antwortet; andere ignorieren
- 7 Bit $\Rightarrow$ theoretisch 128 Slots; praktisch weniger (reservierte Adressen); optional auch 10-Bit-Adressierung

Notizen

Hardware-Kabel durch Software-Adressen ersetzt — wie Namensaufruf im Klassenraum. Nicht jede 7-Bit-Kombination ist frei nutzbar; Details stehen im I2C-Spec und im Sensor-Datenblatt.

Open-Drain

- Problem: zwei Treiber gleichzeitig HIGH und LOW → bei Push-Pull Kurzschlussrisiko
- I2C: Open-Drain / Open-Collector
- Geräte können nur aktiv nach GND (LOW) ziehen
- HIGH: Leitung loslassen (hochohmig) — nicht aktiv „hochdrücken“



Notizen

Mehrere Geräte dürfen gleichzeitig LOW ziehen — Strom fließt nach Masse, ohne Push-Pull-Konflikt.

Pull-Up-Widerstände

- Ohne aktives HIGH: externe Pull-ups nötig (typisch wenige k Ω)
- Je einer an SDA und SCL gegen V_{IO} (z. B. 3,3 V)
- Alle loslassen → Pull-up zieht HIGH (logisch 1)
- Ein Chip zieht → Leitung LOW (logisch 0)

Notizen

Ohne Pull-ups funktioniert I2C physikalisch nicht. HIGH heißt „niemand zieht“; LOW heißt „jemand zieht“.

Dominantes LOW (Wired-AND)

- LOW ist dominant, HIGH rezessiv
- Gleichzeitig 1 und 0 → Leitung wird 0
- Sender kann mitlesen: „Ich wollte 1, Bus ist 0“ → Überstimmt

Notizen

Wired-AND ist Grundlage für ACK und Arbitration bei Multi-Master.

I2C-Nachrichtenaufbau

Typische Transaktion:

1. START (SDA fällt bei SCL HIGH)
2. 7-Bit-Adresse
3. R/W-Bit (0 = Write, 1 = Read)
4. ACK vom Empfänger
5. Datenbyte(s) + ACK/NACK
6. STOP

Feld	Bedeutung
START	Bus belegen, Slaves wecken
Adresse + R/W	Ziel und Richtung
ACK	Quittung
DATA	Nutzdaten

Notizen

START wirkt wie ein Gong: alle hören die Adresse; nur der Treffer bleibt aktiv.

ACK und NACK

- Nach jedem Byte: 9. Takt für Quittung
- Sender lässt SDA los (Tendenz HIGH = NACK)
- Empfänger zieht SDA auf LOW = ACK
- Kein ACK → Gerät fehlt / nicht bereit (Status in Hardware)

Notizen

Vorteil gegenüber SPI: die MCU erkennt fehlende Slaves auf Protokollebene.

Senden vs. Empfangen

Master Write:

- Adresse + R/W = 0 → ACK → Daten vom Master → ACK vom Slave

Master Read:

- Adresse + R/W = 1 → ACK → Slave treibt SDA, Master taktet weiter SCL
- Master sendet ACK/NACK für empfangene Bytes

Notizen

Half-Duplex: R/W wechselt die Richtung auf SDA; SCL bleibt beim Master (außer Stretching).

STOP-Condition

- Master beendet: bei SCL HIGH steigt SDA von LOW nach HIGH
- Signal: Bus frei für neue Transaktionen
- Slaves gehen zurück in Idle

Notizen

Klare Start- und Stop-Signaturen strukturieren den Verkehr auf dem gemeinsamen Medium.

Clock Stretching

- Zu schneller Master-Takt? Slave kann SCL auf LOW halten
- Master will HIGH, sieht aber LOW (Wired-AND) → wartet
- Slave lässt los → Transfer geht weiter

Notizen

Feature, das SPI so nicht hat: der Slave darf den Takt pausieren (z. B. interne Arbeit).

Multi-Master

- Mehrere Master auf demselben Bus möglich
- Beide können START generieren und Sensoren ansprechen
- Risiko: gleichzeitiger Buszugriff → Arbitration nötig

Notizen

I2C erlaubt Oligarchien mehrerer MCUs — SPI ist typisch Single-Master.

Arbitration

- Bei Kollision entscheidet dominantes LOW
- Wer 1 senden will, aber 0 liest, verliert und bricht ab
- Gewinner sendet weiter — Nachricht bleibt intakt

Notizen

Unblutige Kollisionsauflösung: Verlierer wechselt auf Empfang; Gewinner merkt oft nichts.

Nachteile von I2C

- Open-Drain + Pull-up: langsamere Flanken als Push-Pull
- Typisch Standard/Fast Mode: 100 kHz / 400 kHz (vs. SPI: SCK oft einige zehn MHz)
- Buskapazität begrenzt Teilnehmerzahl und Kabellänge

Notizen

Netzwerkfähigkeit kostet Bandbreite. Ideal für Sensoren; Displays eher SPI.

Zusammenfassung der Big 3

Merkmal	UART	SPI	I2C
Leitungen	2	4+	2
Takt	asynchron	synchron	synchron
Routing	Punkt-zu-Punkt	CS / Daisy	adressierter Bus
Speed	niedrig–mittel	SCK oft 10–50 MHz	100–400 kHz
Hardware	Baud, Framing, FIFO	Push-Pull	Open-Drain, Pull-ups
Einsatz	Debug, Module	Display, SD, Flash	Sensornetz

Notizen

Auswahlkriterium: Pins vs. Bandbreite vs. Topologie.

Fragen? — I2C

- Warum verhindert Open-Drain Kurzschlüsse?
- Wie adressiert I2C ohne CS?
- Wie funktioniert ACK/NACK?

Notizen

Als Nächstes: Was, wenn wir die Platine verlassen und lange Kabel in rauen Umgebungen brauchen?

Grenzen von Single-Ended

- UART/SPI/I2C: Single-Ended — Pegel gegen gemeinsames GND
- Lange Kabel / getrennte Massepotentiale → Referenz driftet
- Empfänger sieht verfälschte Spannungen → Bitfehler

Notizen

On-Board (Zentimeter) funktioniert Single-Ended gut. Meterweite Maschinenverkabelung braucht andere Physik.

Elektromagnetische Interferenzen (EMI)

- Lange Leitungen wirken als Antennen
- Motoren/Wechselrichter induzieren Spikes
- Single-Ended: Spike kann 0 als 1 erscheinen lassen

Notizen

Werkshallen und Motorraum sind elektromagnetisch laut — On-Board-Busse sind dafür nicht ausgelegt.

Differentielle Übertragung

- Signal auf zwei Leitungen: normal und invertiert
- Empfänger misst Differenz $V_A - V_B$, nicht gegen GND
- Absolute Masseverschiebung wird irrelevant



Notizen

Logik aus dem Vorzeichen/der Differenz der beiden Adern — nicht aus absolutem Pegel gegen GND.

Twisted Pair

- Adern eng verdreht
- Gleichtaktstörung hebt beide Adern ähnlich an
- Differenz bleibt: $(V_A + \Delta) - (V_B + \Delta) = V_A - V_B$

Notizen

Deshalb Ethernet und viele Feldbusse: Twisted Pair. Störungen heben sich in der Differenz auf.

RS-485

- Physikalische Schicht für lange Strecken (oft mit UART-Logik)
- Transceiver: Single-Ended TX/RX → differenziell A/B
- Reichweiten im dreistelligen Meterbereich möglich
- Häufig mit Protokollen wie Modbus

Notizen

Sie programmieren weiter UART-ähnlich; der Transceiver macht die robuste Leitung.

CAN-Bus

- Controller Area Network (Bosch) — Standard im Fahrzeug
- Differenziell (CAN_H / CAN_L)
- Multi-Drop mit Arbitration (dominant/rezessiv)
- Verbindet Steuergeräte über ein Twisted-Pair

Notizen

CAN kombiniert Bus-Ideen (wie I2C) mit differentieller Robustheit für die Fahrzeugumgebung.

Warum im Labor I2C?

- MCT3-Fokus: Sensoren wenige Zentimeter neben der MCU
- Differentielle Transceiver: Platz, Strom, Kosten
- Für die „letzte Meile“ auf dem Board: I2C oft die passende Wahl

Notizen

Der Exkurs erklärt Feldverkabelung. Zurück aufs Board: I2C-Sensor ansteuern.

Fragen? — Differentiell und Feldbusse

- Wie funktioniert differentielle Übertragung?
- Warum hilft Twisted Pair gegen EMI?
- Wann RS-485/CAN statt I2C?

Notizen

Als Nächstes: Ausblick Labor — Beschleunigungssensor per I2C (Renode).

Ausblick Labor: IMU per I2C

Geplantes Szenario:

- 3-Achsen-Beschleunigungssensor (IMU) am I2C-Bus
- Master: SCL erzeugen, Adresse senden
- Write: Sensor konfigurieren/aufwecken
- Read: Rohwerte X/Y/Z holen

Notizen

Details folgen im Übungsblatt. Ziel: eine vollständige I2C-Transaktion nachvollziehen.

MEMS kurz

- MEMS: mikromechanische Strukturen im Silizium
- Beschleunigung verbiegt Strukturen → Kapazität ändert sich
- Wandlung in Digitalwerte → Register im Sensor
- Diese Register lesen wir per I2C

Notizen

Mechanik im Chip-Maßstab liefert Zahlen; der Bus transportiert sie zur MCU.

Registerkarte des Sensors

- Sensor hat eigene Memory Map (Datenblatt!)
- Beispiele (MPU6050-ähnlich):
 - 0x6B: Power Management
 - 0x3B/0x3C: X-Achse High/Low
- I2C: erst Geräteadresse, dann Registerzeiger, dann Daten

Notizen

Wie ein internes Regal: Adresse des Chips und Adresse des Fachs (Register).

C: I2C initialisieren

Beispielhafte Adressen und Bits (fiktiv — immer Datenblatt prüfen):

```
#define I2C_BASE 0x40010000u
#define I2C_CR1 (*(volatile uint32_t *) (I2C_BASE + 0x00))

void i2c_init(void) {
    I2C_CR1 |= (1u << 0); /* Enable - Bitlage SoC-spezifisch */
    I2C_CR1 |= (1u << 8); /* Master - nur schematisch! */
}
```

- Register-Offsets und Bitpositionen sind plattformspezifisch
- Nicht 1:1 auf einen realen Chip übernehmen

Notizen

Wie Timer/PWM: Steuerung über MMIO. Enable-/Master-Bits hier nur zur Didaktik — im Labor die Referenz Ihres SoCs/Renode-Modells verwenden.

C: Bus-Frequenz

- Typisch 100 kHz oder 400 kHz
- Aus Systemtakt (z. B. 100 MHz) Teiler setzen

```
#define I2C_CLK_DIV (*(volatile uint32_t *) (I2C_BASE + 0x10))

void i2c_set_speed(void) {
    I2C_CLK_DIV = 1000; /* schematisch: 100 MHz / 100 kHz */
}
```

Notizen

Prescaler/Divider so wählen, dass SCL die Spezifikation des Sensors einhält.

C: Write-Transaktion

Ablauf: START → Adresse+W → Reg → Daten → STOP

```
void sensor_wakeup(void) {
    i2c_send_start();
    i2c_send_byte((0x68u << 1) | 0u); /* Addr + Write */
    wait_for_ack();
    i2c_send_byte(0x6B);               /* Register */
    wait_for_ack();
    i2c_send_byte(0x00);               /* Wert */
    wait_for_ack();
    i2c_send_stop();
}
```

Notizen

7-Bit-Adresse um 1 nach links, R/W im LSB. Konkrete API hängt vom Treiber/SoC ab.

C: Read mit Repeated Start

Zuerst Registerzeiger setzen (Write), dann Repeated START und Read:

```
uint8_t read_sensor_x(void) {
    i2c_send_start();
    i2c_send_byte((0x68u << 1) | 0u);
    wait_for_ack();
    i2c_send_byte(0x3B);
    wait_for_ack();

    i2c_send_start(); /* Repeated Start */
    i2c_send_byte((0x68u << 1) | 1u);
    wait_for_ack();
    uint8_t data = i2c_read_byte_nack();
    i2c_send_stop();
    return data;
}
```

Notizen

Klassisches Muster: Pointer setzen, dann lesen — oft ohne STOP dazwischen (Repeated Start).

Folie

74 / 80

Renode: virtueller I2C-Sensor

- Geplant: Sensor als Modell in Renode (ACK/NACK, Register)
- C-Treiber sieht dasselbe Timing-Verhalten wie auf dem Chip
- Validierung ohne Lötarbeit

Notizen

Simulator ersetzt die physische IMU für Treiberentwicklung — Datenblattverhalten nachbilden.

Folie

75 / 80

Platine beschreiben (. repl)

Schematisch:

```
sensor: Sensors.MPU6050 @ i2c0 0x68
```

- Sensor an I2C0, Adresse 0x68
- Verknüpft CPU-I2C mit dem Sensor-Modell

Notizen

Die . repl-Datei beschreibt die virtuelle Verdrahtung — Ihr „LötKolben“ in Textform.

Folie

76 / 80

Stimuli im Startskript (. resc)

Schematisch:

```
machine LoadPlatformDescription @board.repl
sysbus LoadELF @firmware.elf
machine.sensor FeedSample 1.5 0.0 -9.8
start
```

- Virtuelle Beschleunigungswerte einspeisen
- Firmware liest dieselben Registerpfade

Notizen

Laboridee: eingespeiste Vektoren müssen nach korrektem I2C-Read in Ihrer Ausgabe erscheinen.

Folie

77/80

Endianness

- 16-Bit-Werte oft als High- dann Low-Byte (Big-Endian am Sensor)
- In C zusammensetzen:

```
uint8_t hi = read_sensor_x_high();  
uint8_t lo = read_sensor_x_low();  
int16_t x = (int16_t)((hi << 8) | lo);
```

Notizen

Häufiger Fehler: Bytes vertauscht oder Vorzeichen falsch interpretiert.

Folie

78/80

Polling vs. Interrupt

- I2C ist relativ langsam — `wait_for_ack()` ist Polling
- Professionell oft: Transfer starten, auf I2C-IRQ (z. B. über PLIC) warten
- Für Einstiegslabor: Polling ist didaktisch ausreichend

Notizen

Erinnerung Block 3: Polling blockiert; Interrupts entlasten die CPU bei längeren Transfers.

Folie

79/80

Zusammenfassung Block 5

- Seriell ersetzt parallel extern: Pins, Crosstalk, Skew
- UART: asynchron, Punkt-zu-Punkt, Debug
- SPI: synchron, schnell, CS-intensiv
- I2C: adressierter Open-Drain-Bus auf zwei Leitungen
- Differenziell (RS-485, CAN): lange, gestörte Strecken
- Treiber: MMIO, Adresse|R/W, Registerzeiger, Bit-Shifting

Notizen

Damit kann Ihr SoC Module ansprechen, Sensoren lesen und Konsolen nutzen — Grundlage für vernetzte Embedded-Systeme.

Folie

80 / 80

Ausblick und Abschluss

- Übungsblatt: I2C in Renode Schritt für Schritt
- Ziel: erste virtuelle Beschleunigungswerte auslesen

Vielen Dank — viel Erfolg im Labor.

Notizen

Firmware-Entwicklung heißt: Protokollregeln in Registerzugriffe und Timing übersetzen. Viel Erfolg beim Experimentieren.